



Guía de Pruebas Rápidas — Sistema de Venta de Entradas

Objetivo: Poder probar todo el sistema con UN solo comando.



Requisitos Previos

Antes de empezar, comprueba que tienes instalado:

Requisito	Verificar	Instalar
Docker (con compose)	<code>docker compose version</code>	docs.docker.com (https://docs.docker.com/get-docker/)
Python 3.10+	<code>python3 --version</code>	<code>sudo apt install python3</code>
pip	<code>pip --version</code>	<code>sudo apt install python3-pip</code>
NGINX (opcional)	<code>nginx -v</code>	<code>sudo apt install nginx</code>
curl	<code>curl --version</code>	<code>sudo apt install curl</code>



NGINX es opcional. Sin él, las APIs funcionan en `:8000` y `:8001` pero sin balanceo de carga en `:80`.



Instalación Inicial (Solo Primera Vez)

```
# 1. Clonar / situarse en el proyecto
cd Practica_SD

# 2. Instalar dependencias Python
pip install -e .

# 3. Dar permisos de ejecución a los scripts
chmod +x scripts/*.sh

# 4. Verificar que Docker esté corriendo
docker info
```

Verificación rápida:

```
./scripts/status.sh
```

Debería mostrar todos los servicios como “NO DISPONIBLE” (aún no los hemos levantado).



Escenarios de Prueba

Escenario 1: Probar Arquitectura Directa

La arquitectura directa usa **FastAPI + Redis + NGINX** con comunicación síncrona.

```
./scripts/start_direct_only.sh
```

¿Qué levanta?

Servicio	Puerto	Descripción
Redis	6379	Almacén de estado
API Directa #0	8000	Instancia FastAPI
API Directa #1	8001	Instancia FastAPI
NGINX	80	Balanceador (least_conn)

¿Cómo probar?

```
# Health check
curl http://localhost:8000/health

# Compra de entrada (a través de NGINX)
curl -X POST http://localhost/purchase \
  -H "Content-Type: application/json" \
  -d '{"request_id": "test-001", "ticket_type": "unnumbered", "quantity": 1}'

# Ver estado del inventario
curl http://localhost/inventory

# Ejecutar benchmark
./scripts/run_direct_benchmark.sh
```

Parar:

```
./scripts/stop_all.sh
```

Escenario 2: Probar Arquitectura Indirecta

La arquitectura indirecta usa **Gateway + RabbitMQ + Workers** con comunicación asíncrona.

```
./scripts/start_indirect_only.sh
```

¿Qué levanta?

Servicio	Puerto	Descripción
Redis	6379	Almacén de estado
RabbitMQ	5672	Broker de mensajes

RabbitMQ Mgmt	15672	Consola web (guest/guest)
Gateway	8080	Punto de entrada HTTP
Workers x3	—	Procesadores de peticiones

¿Cómo probar?

```
# Health check
curl http://localhost:8080/health

# Compra de entrada
curl -X POST http://localhost:8080/purchase \
  -H "Content-Type: application/json" \
  -d '{"request_id": "test-002", "ticket_type": "unnumbered", "quantity": 1}'

# Consola de RabbitMQ (en navegador)
# http://localhost:15672 → usuario: guest / contraseña: guest

# Ejecutar benchmark
./scripts/run_indirect_benchmark.sh
```

Cambiar número de workers:

```
./scripts/start_indirect_only.sh 5    # 5 workers
```

Parar:

```
./scripts/stop_all.sh
```

Escenario 3: Probar Ambas Arquitecturas

Levanta todo simultáneamente para poder comparar.

```
./scripts/start_all.sh
```

¿Qué levanta?

Todo lo de los escenarios 1 y 2 juntos:

- Directa en :80 (NGINX) / :8000 / :8001
- Indirecta en :8080

Probar las dos:

```
# Directa
curl http://localhost/health
curl http://localhost:8080/health

# Estado general
./scripts/status.sh
```

Escenario 4: Benchmarks Comparativos

Ejecuta el pipeline completo de análisis:

```
# 1. Asegúrate de tener todo levantado
./scripts/start_all.sh

# 2. Ejecutar análisis completo (puede tardar varios minutos)
./scripts/run_full_analysis.sh
```

¿Qué hace?

1. Benchmarks comparativos (directa vs indirecta)
2. Test de escalabilidad (1, 2, 4, 8 workers)
3. Test de contención
4. Genera gráficos y reportes

Resultados:

```
benchmarks/outputs/
├── direct/           ← Resultados arquitectura directa
├── indirect/        ← Resultados arquitectura indirecta
├── comparative/     ← Comparativas
├── scalability/     ← Tests de escalabilidad
├── contention/      ← Tests de contención
├── plots/           ← Gráficos PNG
└── reports/         ← Reportes MD y HTML
```

Benchmarks individuales:

```
# Solo directa
./scripts/reset_system.sh && ./scripts/start_direct_only.sh
./scripts/run_direct_benchmark.sh

# Solo indirecta
./scripts/reset_system.sh && ./scripts/start_indirect_only.sh
./scripts/run_indirect_benchmark.sh

# Con fichero de benchmark personalizado
./scripts/run_direct_benchmark.sh benchmarks/input/benchmark_numbered_60000.txt
```

Comandos Rápidos

Acción	Comando
Levantar todo	<code>./scripts/start_all.sh</code>
Solo directa	<code>./scripts/start_direct_only.sh</code>
Solo indirecta	<code>./scripts/start_indirect_only.sh</code>
Parar todo	<code>./scripts/stop_all.sh</code>
Ver estado	<code>./scripts/status.sh</code>
Ver logs	<code>./scripts/logs.sh</code>
Ver logs directa	<code>./scripts/logs.sh direct</code>
Ver logs workers	<code>./scripts/logs.sh worker</code>
Reset completo	<code>./scripts/reset_system.sh</code>
Benchmark directa	<code>./scripts/run_direct_benchmark.sh</code>
Benchmark indirecta	<code>./scripts/run_indirect_benchmark.sh</code>
Análisis completo	<code>./scripts/run_full_analysis.sh</code>
Health check directa	<code>curl http://localhost:8000/health</code>
Health check indirecta	<code>curl http://localhost:8080/health</code>

Troubleshooting

Puerto ya en uso

Error: Puerto 8000 ya está en uso

Solución:

```
# Ver qué usa el puerto
lsof -i :8000
# Matar el proceso
kill $(lsof -ti :8000)
# O hacer un reset completo
./scripts/stop_all.sh
```

Docker no arranca

Error: Cannot connect to the Docker daemon

Solución:

```
# Verificar que Docker esté corriendo
sudo systemctl start docker
# 0 en macOS: Abrir Docker Desktop
```

Redis connection refused

Error: Connection refused localhost:6379

Solución:

```
# Verificar que el contenedor está corriendo
docker ps | grep redis
# Si no está, reiniciar
./scripts/stop_all.sh
./scripts/start_all.sh
```

RabbitMQ no está listo

Error: RabbitMQ no respondió tras 30s

Solución: RabbitMQ tarda más en arrancar la primera vez (descarga imagen). Esperar e intentar de nuevo:

```
./scripts/stop_all.sh
./scripts/start_all.sh
```

NGINX: permission denied

Error: bind() to 0.0.0.0:80 failed (13: Permission denied)

Solución: El puerto 80 requiere root:

```
# Los scripts ya usan sudo, pero si falla:
sudo nginx -c $(pwd)/deploy/nginx/nginx.conf -p $(pwd)/deploy/nginx/
```

Workers no procesan mensajes

Solución:

```
# Verificar que hay workers corriendo
./scripts/status.sh
# Verificar colas en RabbitMQ
curl -s -u guest:guest http://localhost:15672/api/queues | python3 -m json.tool
```

Quiero empezar de cero

```
# Reset total: para todo, limpia datos, reinicializa
./scripts/reset_system.sh
```

Limpieza

Parar todos los servicios

```
./scripts/stop_all.sh
```

Reset completo (datos + logs + colas)

```
./scripts/reset_system.sh
```

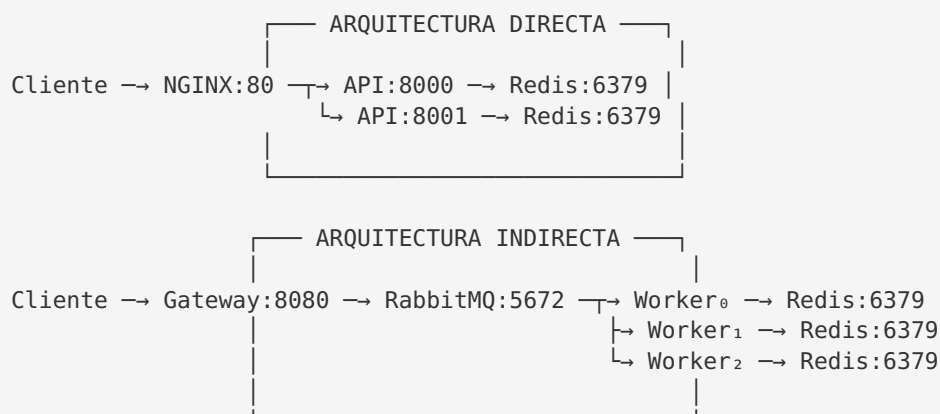
Limpieza profunda (eliminar imágenes Docker)

```
./scripts/stop_all.sh
docker rmi redis:7-alpine rabbitmq:3-management-alpine
```

Verificar que no queda nada

```
./scripts/status.sh
docker ps -a | grep concert
```

Arquitectura del Sistema



Referencias

- **Scripts detallados:** [scripts/README.md](#) (scripts/README.md)
- **Configuración:** [.env](#) (.env) y [config/](#) (config/)
- **Documentación del proyecto:** [docs/](#) (docs/)
- **Docker Compose:** [tools/local_dev/docker-compose.yml](#) (tools/local_dev/docker-compose.yml)
- **NGINX config:** [deploy/nginx/](#) (deploy/nginx/)